Release Notes

Java Card™ 2.1 Specifications

June 7, 1999
Final Revision 1.1



Sun Microsystems, Inc.

901 San Antonio Road

Palo Alto, CA 94043 USA

650 960-1300 fax 650 969-9131

Copyright © 1999 Sun Microsystems, Inc.

901 San Antonio Road, Palo Alto, CA 94303 USA

All rights reserved. Copyright in this document is owned by Sun Microsystems, Inc.

Sun Microsystems, Inc. (SUN) hereby grants to you at no charge a nonexclusive, nontransferable, worldwide, limited license (without the right to sublicense) under SUN's intellectual property rights that are essential to practice the Java Card ™ 2.1 Virtual Machine Specification ("Specification") to use the Specification for internal evaluation purposes only. Other than this limited license, you acquire no right, title, or interest in or to the Specification and you shall have no right to use the Specification for productive or commercial use.

RESTRICTED RIGHTS LEGEND

Use, duplication, or disclosure by the U.S. Government is subject to restrictions of FAR 52.227-14(g)(2)(6/87) and FAR 52.227-19(6/87), or DFAR 252.227-7015(b)(6/95) and DFAR 227.7202-1(a).

SUN MAKES NO REPRESENTATIONS OR WARRANTIES ABOUT THE SUITABILITY OF THE SOFTWARE, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. SUN SHALL NOT BE LIABLE FOR ANY DAMAGES SUFFERED BY LICENSEE AS A RESULT OF USING, MODIFYING OR DISTRIBUTING THIS SOFTWARE OR ITS DERIVATIVES.

TRADEMARKS

Sun, the Sun logo, Sun Microsystems, JavaSoft, JavaBeans, JDK, Java, Java Card, HotJava, HotJava Views, Visual Java, Solaris, NEO, Joe, Netra, NFS, ONC, ONC+, OpenWindows, PC-NFS, EmbeddedJava, PersonalJava, SNM, SunNet Manager, Solaris sunburst design, Solstice, SunCore, SolarNet, SunWeb, Sun Workstation, The Network Is The Computer, ToolTalk, Ultra, Ultracomputing, Ultraserver, Where The Network Is Going, Sun WorkShop, XView, Java WorkShop, the Java Coffee Cup logo, and Visual Java are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.

THIS PUBLICATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT.

THIS PUBLICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN; THESE CHANGES WILL BE INCORPORATED IN NEW EDITIONS OF THE PUBLICATION. SUN MICROSYSTEMS, INC. MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THIS PUBLICATION AT ANY TIME.

Table of Contents

1. Introduction 1

2. Java Card 2.1 Runtime Environment Specification 2

Summary of Corrections 2

Corrections 2

3. Java Card 2.1 Application Programming Interface 6

Summary of Corrections 6

Corrections 6

4. Java Card 2.1 Virtual Machine Specification 11

Summary of Corrections 11

Corrections 11

1. Introduction

This document is the Release Notes for revision 1.1 of the Java Card™ 2.1 Specifications. The specifications include:

- *Java Card™ 2.1 Runtime Environment Specification*
- *Java Card™ 2.1 Application Programming Interface*
- *Java Card™ 2.1 Virtual Machine Specification*

These specifications describe the Java Card 2.1 platform standards of Java Card technology, providing the foundation for designing implementations of Java Card enabled devices and tools.

2. Java Card 2.1 Runtime Environment Specification

The *Java Card 2.1 Runtime Environment Specification* describes the runtime environment required for interoperable Java Card applet execution.

Summary of Corrections

- Section 6.2.8.5 - behavior of `invokeinterface` on JCRE entry point objects specified.
- New section 6.2.8.11 to specify Array object method access behavior.
- Section 8.5 - lifetime of temporary intermediate results in Security and Crypto packages specified.
- Section 10.1.4 - CAP file error checking in Installer specified.
- Section 10.2 - state of selection context during `install()` execution specified.

Corrections

Section 6.2.8.4 Accessing Class Instance Object Methods

Corrected - Erroneous line deleted from first bullet item.

Bytecodes:

`invokevirtual`

- If the object is owned by an applet in the currently active context, then access is allowed.
- Otherwise, if the object is designated a JCRE Entry Point Object, then access is allowed. Context is switched to the object owner's context (JCRE context).
- Otherwise, if JCRE is the currently active context, then access is allowed. Context is switched to the object owner's context.

Final Revision 1.1 Copyright © June 7, 1999 Sun Microsystems

- Otherwise, access is denied.

Original - Erroneous line in first bullet item.

Bytecodes:

`invokevirtual`

- If the object is owned by an applet in the currently active context, then access is allowed. Context is switched to the object owner's context.
- Otherwise, if the object is designated a JCRE Entry Point Object, then access is allowed. Context is switched to the object owner's context (shall be JCRE).
- Otherwise, if JCRE is the currently active context, then access is allowed. Context is switched to the object owner's context.
- Otherwise, access is denied.

Section 6.2.8.5 Accessing Standard Interface Methods

Corrected - Behavior of invokeinterface on JCRE entry point objects specified

Bytecodes:

`invokeinterface`

- If the object is owned by an applet in the currently active context, then access is allowed.
- Otherwise, if the object is designated a JCRE Entry Point Object, then access is allowed. Context is switched to the object owner's context (JCRE context).
- Otherwise, if the JCRE is the currently active context, then access is allowed. Context is switched to the object owner's context.
- Otherwise, access is denied.

Original - underspecified

Bytecodes:

`invokeinterface`

- If the object is owned by an applet in the currently active context, then access is allowed.
- Otherwise, if the JCRE is the currently active context, then access is allowed. Context is switched to the object owner's context.
- Otherwise, access is denied.

Section 6.2.8 Class and Object Access Behavior

Add - new section 6.2.8.11 to specify array object method access behavior

6.2.8.11 Accessing Array Object Methods

Note – The method access behavior of global arrays is identical to that of JCRE Entry Point Objects.

Bytecodes:

`invokevirtual`

- If the array is owned by an applet in the currently active context, then access is allowed.
- Otherwise, if the array is designated a global array, then access is allowed. Context is switched to the array owner's context (JCRE context).
- Otherwise, if JCRE is the currently active context, then access is allowed. Context is switched to the array owner's context.
- Otherwise, access is denied.

Section 8.5 The Security and Crypto packages

Add - Paragraphs to clarify lifetime of temporary storage of intermediate results

The `MessageDigest` object uses temporary storage for intermediate results when the `update()` method is invoked. This intermediate state need not be preserved across power up and reset. The object is reset to the state it was in when previously initialized via a call to `reset()`.

The `Signature` and `Cipher` objects use temporary storage for intermediate results when the `update()` method is invoked. This intermediate state need not be preserved across power up and reset. The object is reset to the state it was in when previously initialized via a call to `init()`.

10.1.4 Installer Behavior

Corrected - Add required CAP file error checks

The JCRE shall guarantee that an applet will *not* be deemed successfully installed if:

- the applet package requires more memory than is available on the card.
- the applet package references a package that is not resident on the card.
- the applet package references another package already resident on the card , but the version of the resident package is not binary compatible with the applet package. For more information on binary compatibility in the Java programming language please see *The Java Language Specification.* Binary compatibility in Java Card technology is discussed in the *Java Card 2.1 Virtual Machine Specification.*
- a class in the applet package is found to contain more package visible virtual methods or instance fields than the limitations enumerated in section 2.2.4.2 of the *Java Card 2.1 Virtual Machine Specification.*

 Final Revision 1.1 Copyright © June 7, 1999 Sun Microsystems

- the applet's `install` method throws an exception before successful return from the `Applet.register` method (see section 3.1)..

Original - incomplete list of installation constraints

The JCRE shall guarantee that an applet will *not* be deemed successfully installed if:

- the applet package must link with another package already resident on the card , but the version of the resident package is not binary compatible with the applet package. For more information on binary compatibility in the Java programming language please see *The Java Language Specification.* Binary compatibility in Java Card technology is discussed in the *Java Card 2.1 Virtual Machine Specification.*

- the applet's `install` method throws an exception before successful return from the `Applet.register` method (see section 3.1).

10.2 The Newly Installed Applet

Add after 2nd paragraph - State of selection context during install() execution

The precise mechanism by which an applet's `install(byte[], short, byte)` method is invoked from the Installer is a JCRE implementer-defined implementation detail. However, there shall be a context switch so that any context-related operations performed by the `install` method (such as creating new objects) are done in the context of the new applet and not in the context of the Installer. The Installer shall also ensure that array objects created during applet class initialization (<clinit>) methods are also owned by the context of the new applet.

The Installer shall ensure that during the execution of the `install()` method, the new applet (not the Installer) is the *currently selected applet*. In addition, any `CLEAR_ON_DESELECT` objects created during the `install()` method shall be associated with the selection context of the new applet.

3. Java Card 2.1 Application Programming Interface

The *Java Card 2.1 Application Programming Interface* defines a set of classes upon which Java Card Applets may be constructed.

Summary of Corrections

- javacard.framework.APDU.waitExtension should be a static method.
- javacard.framework.OwnerPIN.OwnerPIN specifies initial state of validated flag.
- javacard.framework.OwnerPIN.check specifies behavior on VM exception.
- javacard.framework.Util methods use ArrayIndexOutOfBoundException.
- New javacard.security.MessageDigest.reset method to allow MessageDigest object to restart.

Corrections

Class javacard.framework.APDU

Corrected - This should be a static method so that it can be used in methods that don't have a reference to the APDU object, for example install().

waitExtension

`public static void waitExtension() throws APDUException`

Requests additional processing time from CAD. The implementation should ensure that this method needs to be invoked only under unusual conditions requiring excessive processing times.

Notes:

- *In T=0 protocol, a NULL procedure byte is sent to reset the work waiting time (see ISO 7816-3).*

Final Revision 1.1 Copyright © June 7, 1999 Sun Microsystems

- *In T=1 protocol, the implementation needs to request the same T=0 protocol work waiting time quantum by sending a T=1 protocol request for wait time extension(see ISO 7816-3).*
- *If the implementation uses an automatic timer mechanism instead, this method may do nothing.*

Throws: `APDUException`
with the following reason codes:
- `APDUException.ILLEGAL_USE` if `setOutgoingNoChaining()` previously invoked.
- `APDUException.IO_ERROR` on I/O error.

Original - instance method

waitExtension
`public void waitExtension() throws APDUException`
Requests additional processing time from CAD. The implementation should ensure that this method needs to be invoked only under unusual conditions requiring excessive processing times.
Notes:
- *In T=0 protocol, a NULL procedure byte is sent to reset the work waiting time (see ISO 7816-3).*
- *In T=1 protocol, the implementation needs to request the same T=0 protocol work waiting time quantum by sending a T=1 protocol request for wait time extension(see ISO 7816-3).*
- *If the implementation uses an automatic timer mechanism instead, this method may do nothing.*

Throws: `APDUException`
with the following reason codes:
- `APDUException.ILLEGAL_USE` if `setOutgoingNoChaining()` previously invoked.
- `APDUException.IO_ERROR` on I/O error.

Class javacard.framework.OwnerPIN

Corrected - Specifies initial state of validated flag.

OwnerPIN
`public OwnerPIN(byte tryLimit, byte maxPINSize) throws PINException`
Constructor. Allocates a new `PIN` instance with validated flag set to false.
Parameters:
`tryLimit` - the maximum number of times an incorrect `PIN` can be presented.
`maxPINSize` - the maximum allowed `PIN` size. `maxPINSize` must be >=1.
Throws: `PINException`
with the following reason codes:
- `PINException.ILLEGAL_VALUE` if `maxPINSize` parameter is less than 1.

Original - underspecified

OwnerPIN
`public OwnerPIN(byte tryLimit, byte maxPINSize) throws PINException`

Constructor. Allocates a new `PIN` instance.

Parameters:

`tryLimit` - the maximum number of times an incorrect `PIN` can be presented.

`maxPINSize` - the maximum allowed `PIN` size. `maxPINSize` must be >=1.

Throws: PINException

with the following reason codes:

- `PINException.ILLEGAL_VALUE` if `maxPINSize` parameter is less than 1.

 Final Revision 1.1 Copyright © June 7, 1999 Sun Microsystems

Class javacard.framework.OwnerPIN

Interface javacard.framework.PIN

Corrected - Specifies behavior on VM exception

check

`public boolean check(byte[] pin, short offset, byte length) throws ArrayIndexOutOfBoundsException, NullPointerException`

Compares `pin` against the `PIN` value. If they match and the `PIN` is not blocked, it sets the validated flag and resets the try counter to its maximum. If it does not match, it decrements the try counter and, if the counter has reached zero, blocks the `PIN`. Even if a transaction is in progress, internal state such as the try counter, the validated flag and the blocking state must not be conditionally updated.

Notes:

- *If* `NullPointerException` *or* `ArrayIndexOutOfBoundsException` *is thrown, the validated flag must be set to* `false`*, the try counter must be decremented and, the* `PIN` *blocked if the counter reaches zero.*
- *If* `offset` *or* `length` *parameter is negative an* `ArrayIndexOutOfBoundsException` *exception is thrown.*
- *If* `offset+length` *is greater than* `pin.length`*, the length of the* pin *array, an* `ArrayIndexOutOfBoundsException` *exception is thrown.*
- *If* `pin` *parameter is* `null` *a* `NullPointerException` *exception is thrown.*

Parameters:

`pin` - the byte array containing the PIN value being checked
`offset` - the starting offset in the pin array
`length` - the length of `pin`.

Returns:

`true` if the PIN value matches; `false` otherwise

Throws:

`ArrayIndexOutOfBoundsException` - if the check operation would cause access of data outside array bounds.
`NullPointerException` - if `pin` is `null`

Original - underspecified

check

`public boolean check(byte pin[], short offset, byte length)`

Compares pin against the `PIN` value. If they match and the `PIN` is not blocked, it sets the validated flag and resets the try counter to its maximum. If it does not match, it decrements the try counter, and if the counter has reached zero, blocks the `PIN`.

Parameters:

`pin` - the byte array containing the `PIN` value being checked
`offset` - the starting offset in the `pin` array
`length` - the length of `pin`.

Returns:

`true` if the `PIN` value matches; `false` otherwise

Class javacard.framework.Util

Corrected – general use of out of bounds exception

For the methods that use arrays change all uses of `IndexOutOfBoundsException` to `ArrayIndexOutOfBoundsException`.

Class javacard.security.MessageDigest

Add - This method is required to allow `MessageDigest` object to restart

reset

```
public abstract void reset()
```

Resets the `MessageDigest` object to the initial state for further use.

4. Java Card 2.1 Virtual Machine Specification

The *Java Card 2.1 Virtual Machine Specification* defines the required behavior of a virtual machine for a Java Card enabled device.

Summary of Corrections

- Addition of a structure in the export file to enumerate public superclasses and public superinterfaces of a class or interface, respectively. This information is required when verifying type casts in bytecodes from a class or interface to one of its superclasses or superinterfaces, respectively. See sections **5.4**, **5.4.2**, and **5.5**.
- Modification of the superinterfaces array in interface_info structure in the CAP file to include all superinterfaces of the direct superinterfaces. This modification decreases the complexity and amount of functionality required by the checkcast and instanceof bytecodes. See section **6.8.1**.
- All other changes represent minor corrections and clarifications. See sections **5.5**, **5.6**, **6.4**, **6.5**, **6.8.1**, **6.12**, and **6.13.1**.

Corrections

Section 5.4 Constant Pool

Corrected - Generalize name of constant type to include both classes and interfaces.

Constant Type	Value
CONSTANT_Package	13
CONSTANT_Classref	7
CONSTANT_Integer	3
CONSTANT_UTf8	1

Table 5.1 Export file constant pool tags

Original -

Constant Type	Value
CONSTANT_Package	13
CONSTANT_Interfaceref	7
CONSTANT_Integer	3
CONSTANT_UTf8	1

Table 5.1 Export file constant pool tags

Section 5.4.2 CONSTANT_Interfaceref

Corrected - Generalize name of constant type to include both classes and interfaces.

5.4.2 CONSTANT_Classref

The CONSTANT_Classref_info structure is used to represent a class or interface:

```
CONSTANT_Classref_info {
        u1 tag
        u2 name_index
}
```

The items of the CONSTANT_Classref_info structure are the following:

tag

The tag item has the value of CONSTANT_Classref (7).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (§ 5.4.4) structure representing a valid fully qualified Java class or interface name. This name is fully qualified since it may represent a class or interface defined in a package other than the one described in this export file.

As in Java class files, ASCII periods (‘.’) that normally separate the identifiers in a class or interface name are replaced by ASCII forward slashes (‘/’). For example, the interface name javacard.framework.Shareable is represented in a CONSTANT_Utf8_info structure as javacard/framework/Shareable.

Original -

5.4.2 CONSTANT_Interfaceref

The CONSTANT_Interfaceref_info structure is used to represent an interface:

```
CONSTANT_Interfaceref_info {
        u1 tag
        u2 name_index
```

}

The items of the CONSTANT_Interfaceref_info structure are the following:

tag

The tag item has the value of CONSTANT_Interface (7).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (§5.4.4) structure representing a valid fully qualified Java interface name. These names are fully qualified because they may be defined in a package other than the one described in the export file.

As in Java class files, ASCII periods ('.') that normally separate the identifiers in a class or interface name are replaced by ASCII forward slashes ('/'). For example, the interface name javacard.framework.Shareable is represented in a CONSTANT_Utf8_info structure as javacard/framework/Shareable.

Section 5.5 Classes and Interfaces

Corrected - Modify type of export_interfaces_count, and add public superclass array for verifying an implied cast to a superclass.

```
class_info {
        u1 token
        u2 access_flags
        u2 name_index
        u2 export_supers_count
        u2 supers[export_supers_count]
        u1 export_interfaces_count
        u2 interfaces[export_interfaces_count]
        u2 export_fields_count
        field_info fields[export_fields_count]
        u2 export_methods_count
        method_info methods[export_methods_count]
}
```

Original -

```
class_info {
        u1 token
        u2 access_flags
        u2 name_index
        u2 export_interfaces_count
        u2 interfaces[export_interfaces_count]
        u2 export_fields_count
        field_info fields[export_fields_count]
        u2 export_methods_count
        method_info methods[export_methods_count]
}
```

Section 5.5 Classes and Interfaces, name_index

Corrected - Modify name_index type to use new CONSTANT_Classref type.

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Classref_info (§5.4.2) structure representing a valid, fully qualified Java class or interface name.

Original -

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (§ 5.4.4) structure representing a valid Java class name stored as a simple (not fully qualified) name, that is, as a Java identifier.[1]

Section 5.5 Classes and Interfaces, superclass

Corrected - Add public superclass array for verifying an implied cast to a superclass.

export_supers_count

The value of the export_supers_count item indicates the number of entries in the supers array.

supers[]

The super array contains an entry for each public superclass of this class or interface. It does not include package visible superclasses.

Each value in the supers array must be a valid index into the constant_pool table. The constant_pool entry at each value must be a CONSTANT_Classref_info (§5.4.2) structure representing a valid, fully qualified Java class or interface name. Entries in the supers array may occur in any order.

Section 5.5 Classes and Interfaces, interfaces

Corrected - Modify interfaces array to use new CONSTANT_Classref type, and remove order restrictions.

interfaces[]

The interfaces array contains an entry for each public interface implemented by this class or interface. It does not include package visible interfaces. It does include all public superinterfaces in the hierarchies of public interfaces implemented by this class or interface.

---

1 In Java class files class names are fully qualified. In Java Card export files all classes and interfaces enumerated are defined in the package of the export file making it unnecessary for class names to be fully qualified.

Each value in the interfaces array must be a valid index into the constant_pool table. The constant_pool entry at each value must be a CONSTANT_Classref_info (§5.4.2) structure representing a valid, fully qualified Java interface name. Entries in the interfaces array may occur in any order.

Original -

interfaces[]

The interfaces array contains an entry for each public interface implemented by this class or interface. It does not include package visible interfaces. It does include all public superinterfaces in the hierarchy of public interfaces implemented by this class or interface.

Each value in the interfaces array must be a valid index into the constant_pool table. The constant_pool entry at each value of interfaces[i], where 0 <= i < export_interfaces_count, must be a CONSTANT_Interfaceref_info structure representing an interface which is an public superinterface of this class or interface type, in the left-to-right order given in the source for the type and its superclasses or superinterfaces.

Section 5.6 Fields

Corrected - Change token item value for compile-time constants.

token

The token item is the token assigned to this field. There are three scopes for field tokens: final static fields of primitive types (compile-time constants), all other static fields, and instance fields.

If this field is a compile-time constant, the value of the token item is 0xFF.

Original -

token

The token item is the token assigned to this field. There are three scopes for field tokens: final static fields of primitive types (compile-time constants), all other static fields, and instance fields.

If this field is a compile-time constant, the value of the token item is 0xFFFF.

Section 6.4 Directory Component

Corrected - Modify type of size item in custom_component_info structure.

```
custom_component_info {
        u1 component_tag
        u2 size
        u1 AID_length
        u1 AID[AID_length]
}
```

Original -

```
custom_component_info {
        u1 component_tag
```

```
        u1 size
        u1 AID_length
        u1 AID[AID_length]
}
```

Section 6.5 Applet Component

Corrected - Clarified restriction to value of count item.

count

The count item indicates the number of applets defined in this package. The value of the count item must be greater than zero.

Original -

count

The count item indicates the number of applets defined in this package.

Section 6.8.1 interface_info and class_info, first reference item name

Corrected - Modify item name in class_info structure.

```
class_info {
    u1 bitfield {
            bit[4] flags
            bit[4] interface_count
    }
    class_ref super_class_ref
    u1 declared_instance_size
    u1 first_reference_token
    u1 reference_count
    u1 public_method_table_base
    u1 public_method_table_count
    u1 package_method_table_base
    u1 package_method_table_count
    u2 public_virtual_method_table[public_method_table_count]
    u2 package_virtual_method_table[package_method_table_count]
    implemented_interface_info interfaces[interface_count]
}
```

Original -

```
class_info {
    u1 bitfield {
            bit[4] flags
            bit[4] interface_count
    }
    class_ref super_class_ref
    u1 declared_instance_size
    u1 first_reference_index
    u1 reference_count
    u1 public_method_table_base
```

```
    u1 public_method_table_count
    u1 package_method_table_base
    u1 package_method_table_count
    u2 public_virtual_method_table[public_method_table_count]
    u2 package_virtual_method_table[package_method_table_count]
    implemented_interface_info interfaces[interface_count]
}
```

Section 6.8.1 interface_info and class_info, interface_count and interfaces definition

Corrected - Modify superinterfaces array in interface_info structure to include all superinterfaces of the direct superinterfaces.

interface_count

The interface_count item of the interface_info structure indicates the number of entries in the superinterfaces table item. The value represents the number of direct and indirect superinterfaces of this interface. Indirect superinterfaces are the set of superinterfaces of the direct superinterfaces.Valid values are between 0 and 15, inclusive.

The interface_count item of the class_info structure indicates the number of entries in the interfaces table item. The value represents the number of interfaces implemented by this class, including superinterfaces of those interfaces and potentially interfaces implemented by superclasses of this class. Valid values are between 0 and 15, inclusive.

superinterfaces

The superinterfaces item of the interface_info structure is an array of class_ref structures representing the superinterfaces of this interface. The class_ref structure is defined as part of the CONSTANT_Classref_info structure (§6.7.1). This array is empty if this interface has no superinterfaces. Both direct and indirect superinterfaces are represented in the array.

Original -

interface_count

The interface_count item of the interface_info structure indicates the number of entries in the superinterfaces table item. The value represents the number of immediate superinterfaces of this interface.It does not include superinterfaces of the superinterfaces. Valid values are between 0 and 15, inclusive.

The interface_count item of the class_info structure indicates the number of entries in the interfaces table item. The value represents the number of interfaces implemented by this class, including superinterfaces of those interfaces and potentially interfaces implemented by superclasses of this class. Valid values are between 0 and 15, inclusive.

superinterfaces

The superinterfaces item of the interface_info structure is an array of class_ref structures representing the superinterfaces of this interface. The class_ref structure is defined as part of the CONSTANT_Classref_info structure (§6.7.1). This array is empty if this interface has no superinterfaces. Only immediate superinterfaces are represented in the array. Superinterfaces of superinterfaces are not included, and class Object is not included either.

Section 6.12 Export Component

Corrected - Clarified restriction to value of class_count item.

class_count

The class_count item represents the number of entries in the class_exports table. The value of the class_count item must be greater than zero.

Original -

class_count

The class_count item represents the number of entries in the class_exports table.

Section 6.13.1 class_descriptor_info

Corrected - Modify the type of item referenced by elements in the interfaces[] array item.

interfaces[]

The interfaces item represents an array of interfaces implemented by this class or interface. The elements in the array are class_ref structures indicating the location of the interface_info structure in the Class Component (§6.8). The class_ref structure is defined as part of the CONSTANT_Classref_info structure (§6.7.1).

Original -

interfaces[]

The interfaces item represents an array of interfaces implemented by this class or interface. The elements in the array are class_ref structures indicating the location of the class_info structure in the Class Component (§6.8). The class_ref structure is defined as part of the CONSTANT_Classref_info structure (§6.7.1).

Final Revision 1.1 Copyright © June 7, 1999 Sun Microsystems